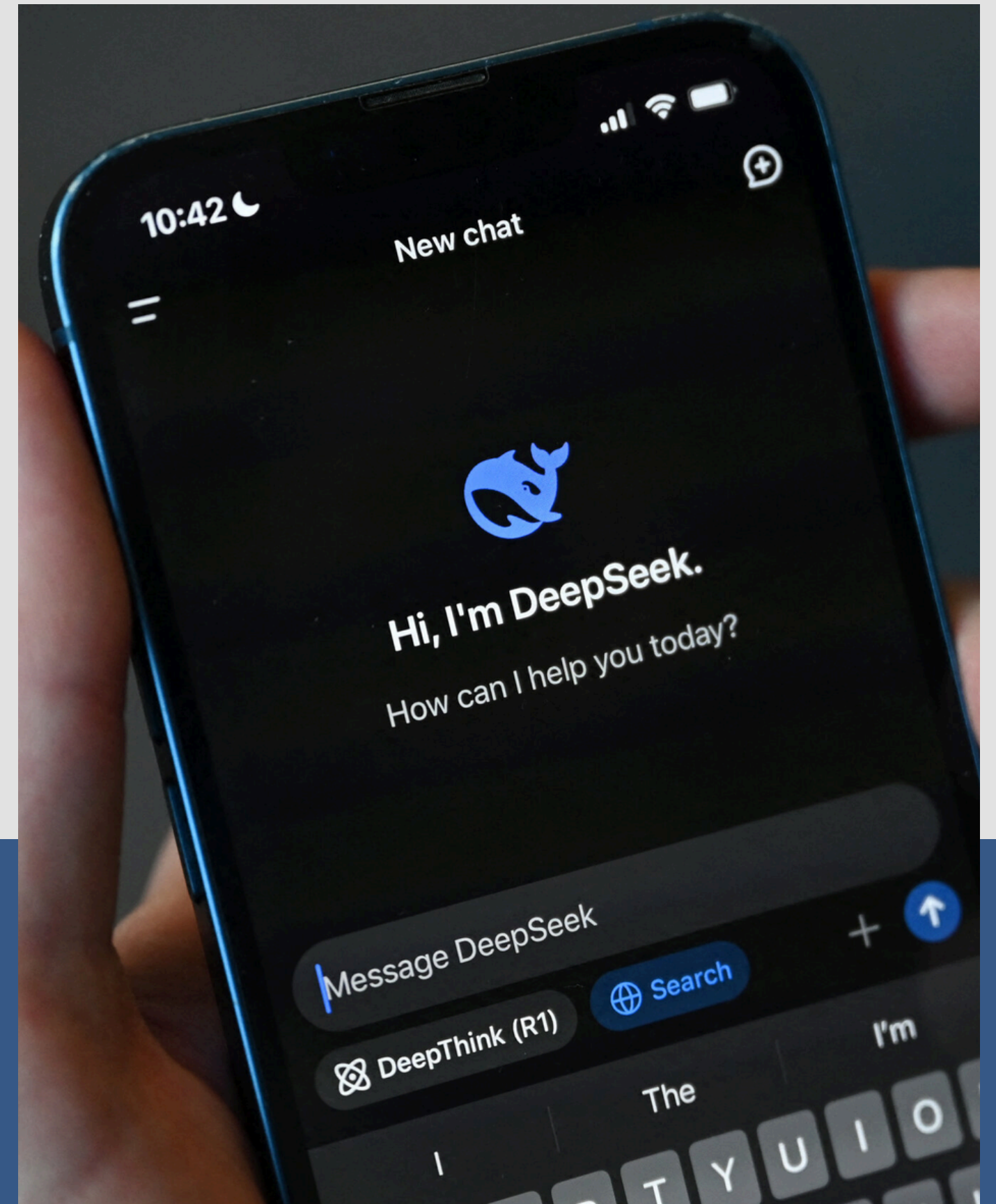
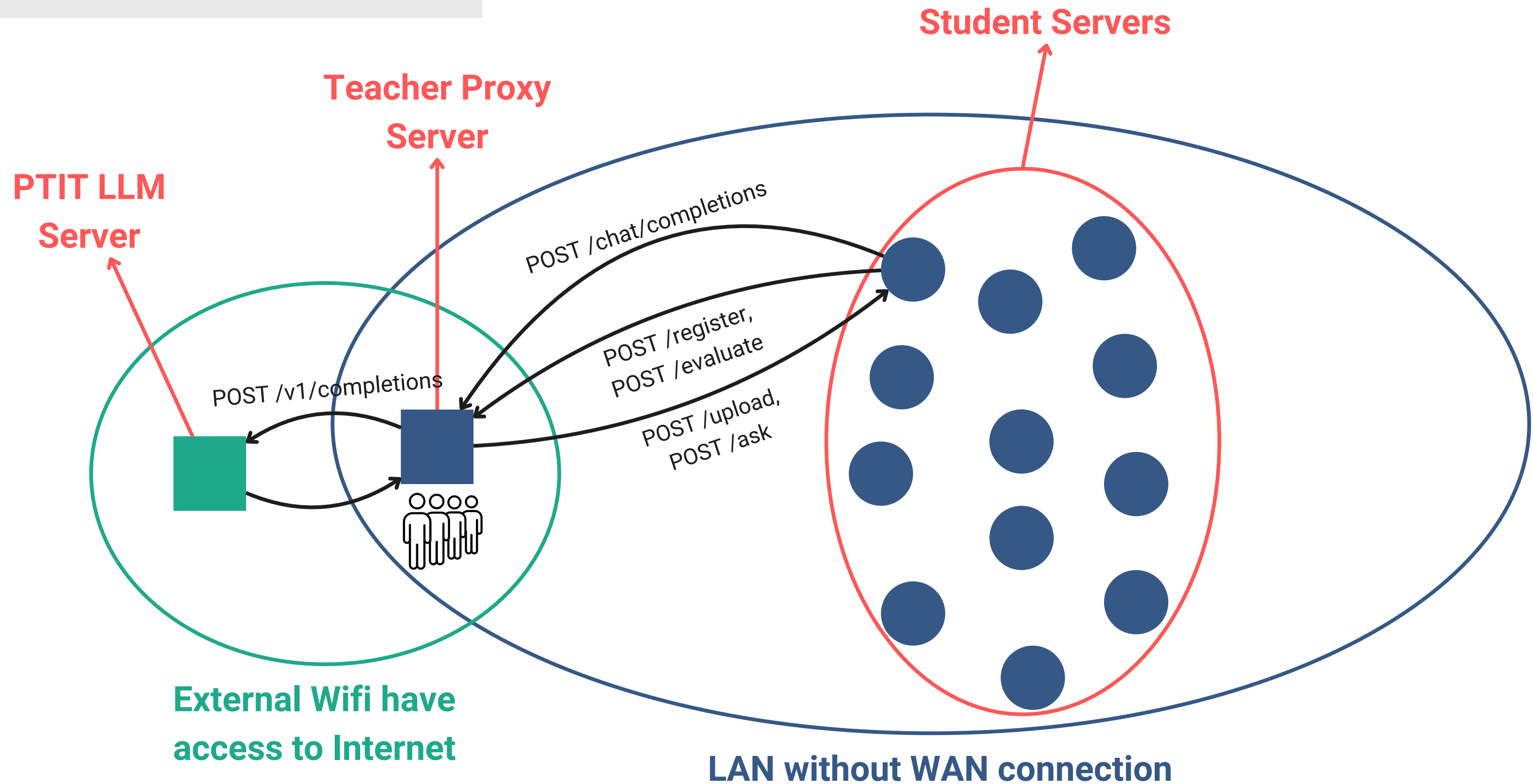

BÀI TẬP: Xây dựng hệ thống cung cấp Api Endpoint để truy vấn tài liệu sử dụng LLM + RAG với framework FastAPI

● ● ● ● ●



Luồng thi



Proxy Competition Server

Backend cho việc tổ chức thi cuối kì Offline RAG (Retrieval-Augmented Generation) Competition.

Hệ thống đóng vai trò:

- API Gateway trung gian cho toàn bộ cuộc thi
- Proxy điều phối request tới Public LLM
- Hệ thống chấm điểm tự động
- Bộ điều phối timeout và queue chống rate-limit

Các thành phần chính

Teacher Server

- Quản lý cuộc thi
- Gửi document
- Gửi câu hỏi
- Chấm điểm
- Proxy LLM API

Student Server (Máy sinh viên)

- Nhận tài liệu
- Chunking + VectorDB
- Thực hiện RAG
- Trả lời câu hỏi

Teacher Server APIs

Teacher Server Base url

`http://192.168.50.218:8000/api/v1`

POST /competition/register

Chức năng: Sinh viên đăng ký địa chỉ Student Server

Header bắt buộc để định danh sinh viên:

X-Student-ID: <Mã Sinh viên viết hoa>

Request Schema:

```
class RegisterPayload(BaseModel):  
    server_url: str
```

Response Schema:

```
class RegisterResponse(BaseModel):  
    status: str  
    student_id: str  
    server_url: str
```

Ví dụ Request:

```
{  
    "server_url": "http://192.168.1.15:5000"  
}
```

Ví dụ Response:

```
{  
    "message": "Đăng ký thành công!",  
    "student_id": "B21DCCN629",  
    "server_url": "http://192.168.1.15:5000"  
}
```

Teacher Server APIs

Teacher Server Base url

<http://192.168.50.218:8000/api/v1>

POST /competition/evaluate

Chức năng: Bắt đầu quá trình thi, Teacher Server sẽ tự động gọi /upload và sau đó gọi 10 lần /ask

Header bắt buộc: X-Student-ID: <Mã Sinh viên viết hoa>

Request Schema: Không cần Content Body.

Response Schema:

```
class EvaluateResponse(BaseModel):
```

```
    student_id: str
```

```
    score: float
```

```
    status: str
```

```
    detail: list
```

Ví dụ Response (Sau khi kết thúc quá trình):

```
{
  "student_id": "B21DCCN629",
  "score": 8.0,
  "status": "completed",
  "detail": [ ... ]
}
```

POST /competition/reset

Chức năng: Xóa bỏ điểm cũ, reset trạng thái thi để bắt đầu lại (nếu code bị crash).

Header bắt buộc: X-Student-ID: <Mã Sinh viên viết hoa>

Request Schema: Không cần Content Body.

Response Schema:

```
class ResetResponse(BaseModel):
```

```
    status: str
```

```
    message: str
```

Ví dụ Response:

```
{
  "status": "success",
  "message": "Đã reset trạng thái cho sinh viên <Mã Sinh viên viết hoa>"
}
```


Teacher Server APIs

Teacher Server Base url

<http://192.168.50.218:8000/api/v1>

GET /competition/result

Chức năng: Kiểm tra trạng thái và điểm hiện tại của sinh viên.

Header bắt buộc: X-Student-ID: <Mã Sinh viên viết hoa>

Request Schema: Không cần Content Body.

Response Schema:

```
class ResultResponse(BaseModel):  
    student_id: str  
    score: float  
    status: str  
    current_question: int
```

Ví dụ Response (Sau khi kết thúc quá trình):

```
{  
    "student_id": "B21DCCN629",  
    "score": 5.0,  
    "status": "evaluating",  
    "current_question": 6  
}
```

Không có mạng gọi LLM thế nào?

Gọi Proxy LLM qua proxy server để sinh Text

POST /proxy

```
from openai import OpenAI

client = OpenAI(
    base_url="http://192.168.50.218:8000/api/v1/proxy",
    api_key="B21DCCN629" # Dùng MSSV trên lớp làm API KEY
)

res = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "..."}]
)
```

Teacher Server như một Client

Teacher Server KHÔNG chỉ
ngồi chờ Request. Sau khi
Student gọi `/evaluate`,
**Teacher Server sẽ chủ động
gửi Request** ngược về Student
Server.

1. Gửi dữ liệu tài liệu RAG gốc
xuống máy sinh viên. Đợi
tối đa 120 giây
2. Bơm 10 câu hỏi lần lượt. Đợi
mỗi câu tối đa 60 giây.

Ví dụ Request đến /upload:

```
{  
  "doc_id": "none",  
  "text": "Nội dung tài liệu RAG..."  
}
```

Ví dụ Request đến /ask:

```
{  
  "question": "RAG là gì? A.xxx B.xxx C.xxx D.xxx"  
}
```


Student Server API Endpoints

Sinh viên **BẮT BUỘC** phải viết đúng 2 endpoint /upload, /ask với schema đã thống nhất từ trước trên server local của mình

POST /upload

Chức năng: Nhận document từ Teacher Server. Thực hiện Chunking, Embedding và lưu vào VectorDB ở đây.

Request Schema:

```
class UploadRequest(BaseModel):  
    doc_id: Optional[str] = None  
    text: str
```

Response Schema:

```
class UploadResponse(BaseModel):  
    status: str  
    doc_id: Optional[str] = None  
    chunks: int
```

Ví dụ Response:

```
{  
    "status": "success",  
    "doc_id": "abc_doc",  
    "chunks": 42  
}
```

Student Server API Endpoints

Sinh viên **BẮT BUỘC** phải viết đúng 2 endpoint /upload, /ask với schema đã thống nhất từ trước trên server local của mình

POST /ask

Chức năng: Nhận truy vấn, retrieve context từ VectorDB, gửi cho LLM và trả ra đáp án trắc nghiệm.

Request Schema:

```
class AskRequest(BaseModel):  
    question: str
```

Response Schema:

```
class AskResponse(BaseModel):  
    answer: str  
    sources: List[str] = []
```

Ví dụ Response:

```
{  
    "answer": "B",  
    "sources": [  
        "chunk_1_content...",  
        "chunk_2_content..."  
    ]  
}
```

Quy định

Giống như điền báp án trắc nghiệm,
`answer` **BẮT BUỘC** chỉ được là 1 ký tự duy
nhất: A / B / C / D

POST /ask

Chức năng: Nhận truy vấn, retrieve context từ VectorDB, gửi cho LLM và trả ra đáp án trắc nghiệm.

Request Schema:

```
class AskRequest(BaseModel):  
    question: str
```

Response Schema:

```
class AskResponse(BaseModel):  
    answer: str  
    sources: List[str] = []
```

Ví dụ Response:

```
{  
    "answer": "B",  
    "sources": [  
        "chunk_1_content...",  
        "chunk_2_content..."  
    ]  
}
```



Thank You

